Semester Project — End-to-End Data Engineering Pipeline

1. Goal

Design and implement a small, cloud-based data engineering solution that transforms raw data from multiple sources into a useful, queryable **data product**.

Your solution should demonstrate the main stages of a data engineering workflow:

Data Sources → Ingestion → Raw Data → Processing → Curated Data → Data Product

Data quality, basic reliability, and documentation are also part of the project.

The goal is not to build a production-grade data platform. Focus on creating a working, understandable, and reproducible solution using the concepts and technologies introduced during the semester.

You may reuse and adapt examples from the labs, documentation, and AI assistants, provided that you understand your implementation and can explain it during the project demonstration.

2. What You Need to Build

Step 1 — Choose Your Data and Analytical Goal

Select at least **two public data sources** that can be meaningfully combined to answer an analytical question.

For example:

- NYC Taxi trip data and weather data: How does weather affect taxi demand?
- Public transport schedules and weather data: How do transport patterns vary with weather conditions?
- Statistical indicators from different sources: How have selected economic or social indicators evolved?

You can find suitable datasets through:

- NYC Taxi Data
- Open-Meteo
- Eurostat
- BKK Open Data
- EU Open Data
- Kaggle

Your sources may use different input formats, such as CSV, JSON, Parquet, or REST APIs.

Requirements:

- Define a clear analytical question.
- Use data from both sources in your final result.
- Briefly describe the sources, their structure, and how they can be combined.
- Note any relevant data limitations, update frequency, or privacy considerations.

You are encouraged to work with a **small, manageable subset** of the available data. Processing large datasets is not required.

Step 2 — Build the Data Pipeline

Implement a pipeline consisting of three logical data layers:

Layer	Purpose	Example operations
Raw	Preserve the ingested source data	Download files, collect API responses

Layer	Purpose	Example operations
Staged	Prepare data for further processing	Validate schemas, handle missing values, convert data types
Curated	Produce data suitable for analysis	Join datasets, calculate aggregates, create analytical tables

Use **PySpark** for at least one meaningful transformation, such as cleaning, joining, enriching, or aggregating data.

Store the processed data in a suitable analytical format, such as **Parquet or Delta Lake**.

The layers may be implemented as separate folders, tables, or storage locations. They do not need separate storage services.

Your solution must produce at least one curated dataset that:

- Combines information from both data sources.
- Supports your analytical goal.
- Can be queried using SQL.

For example, a curated table might contain daily taxi trip counts, average trip distances, and corresponding weather conditions.

Step 3 — Add Data Quality and Basic Reliability

Implement a small set of meaningful **data quality checks** appropriate for your dataset.

Examples include:

- Required fields must not contain null values.
- Numeric values must fall within reasonable ranges.
- Key fields must be unique where appropriate.
- Records from different sources must match according to your chosen join logic.

You may implement these checks directly in Python/PySpark or use a data quality framework.

At least one check should produce a visible failure or warning when invalid data is detected.

Your pipeline should also be **safely repeatable**: running it again with the same input should not unintentionally duplicate records or corrupt the results.

Basic logging or clear execution messages should make it possible to identify whether the main processing steps succeeded.

Step 4 — Run the Solution in the Cloud

Azure is the recommended platform, as it is used in the labs. AWS and GCP are also acceptable.

Choose a reasonably simple architecture that supports the following components:

Component	What is needed	Possible choices
Storage	Raw, staged, and curated datasets	Azure Blob Storage, ADLS, or equivalent
Processing	Run PySpark transformations	Spark on a VM, Databricks, Fabric, or Synapse
Pipeline execution	Run processing steps in the correct order	Python script, notebook workflow, or orchestration service
SQL access	Query the curated dataset	Spark SQL, Databricks SQL, or another compatible query engine

Keep the architecture as simple as possible. You do not need a separate cloud service for every component.

You may use the same environment for multiple purposes, such as processing data and executing SQL queries.

Infrastructure as Code, containers, and Kubernetes are **not required**. Manually created resources are acceptable.

The infrastructure does not need to be recreated automatically. However, you must document the resources and configuration needed to run the solution.

Step 5 — Make the Pipeline Reproducible

Your pipeline must have a clearly documented way to execute its processing steps, preferably from a single command, script, or workflow.

Running the pipeline should not require manually repeating a long sequence of configuration steps.

You should be able to demonstrate that the solution works with an existing cloud environment, including:

1. Starting the pipeline.
2. Observing the processing steps and quality checks.
3. Inspecting the resulting curated dataset.
4. Executing a SQL query against the final data product.

You do not need to recreate the cloud infrastructure during the demonstration.

You may use a small, previously downloaded input snapshot for the live demonstration, especially when external APIs or large downloads would make execution unreliable. Your complete solution must still include or document how the original data is ingested.

The demonstration must execute the actual transformations and validations, rather than merely display previously generated results.

3. Example Implementations

The following two examples illustrate how the semester project could be implemented using different levels of infrastructure and automation.

These are examples, not mandatory architectures. You may follow either approach, combine elements from them, or design your own solution.

Both examples solve the same analytical problem but use different technologies and levels of automation.

Neither architecture is automatically considered better. Evaluation is based on the correctness, reliability, and quality of your implementation, as well as your understanding of the solution.

Common Example — NYC Taxi Trips and Weather

Analytical question: How do daily taxi trip volumes vary with weather conditions in New York City?

Data sources:

- NYC Taxi Trip Records: Yellow taxi trips for January 2024, in Parquet format.
- Open-Meteo Historical Weather API: Daily precipitation and temperature for New York City during the same period.

The pipeline produces a curated dataset with the following structure:

date	trip_count	avg_trip_distance	precipitation_mm
2024-01-01	...	...	...
2024-01-02	...	...	...

Here, `trip_count` is the number of taxi trips per day, `avg_trip_distance` is their average recorded distance, and `precipitation_mm` represents daily precipitation.

This dataset allows users to compare taxi activity on dry and rainy days.

For simplicity, use one representative weather location for the city. This is a city-level approximation, not a precise weather measurement for every individual trip.

A subset of the taxi records is sufficient for this project.

Example A — Simple, Script-Based Pipeline

Approach: Use basic cloud storage, a single virtual machine, Python scripts, and PySpark. No dedicated orchestration platform is required.

Technologies:

- Azure Blob Storage for data storage.
- One Azure Linux VM for execution.
- Python for ingestion and pipeline execution.
- PySpark running in local mode for data processing.
- Spark SQL for querying the final dataset.

Architecture:

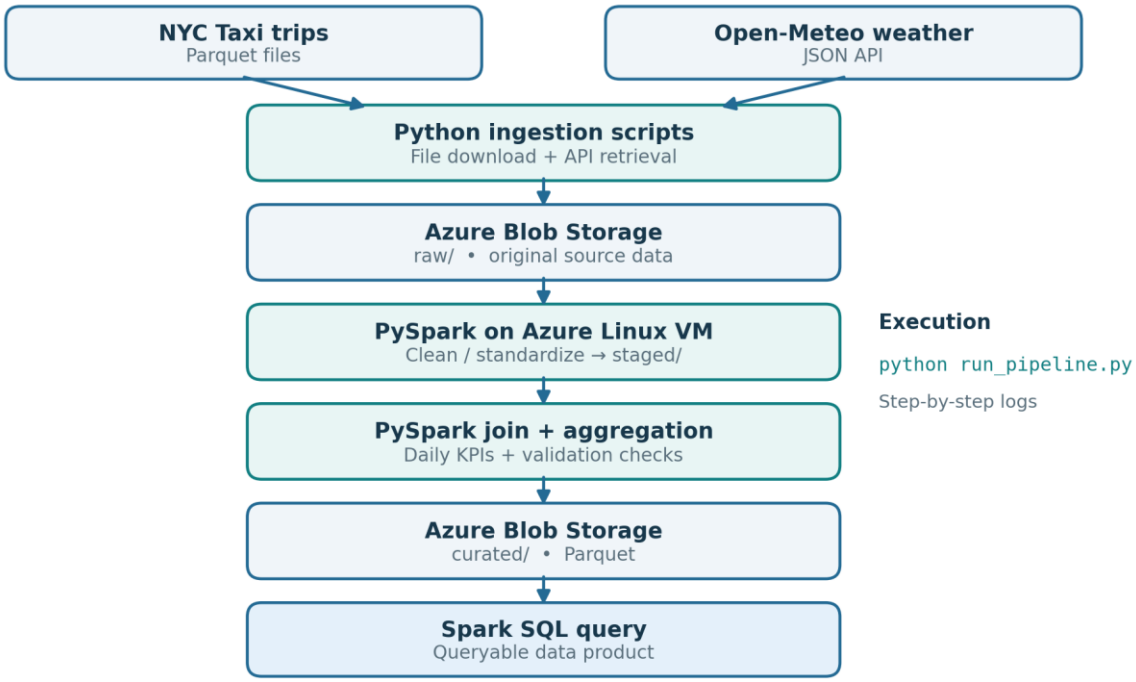


Figure 1. Script-based Azure pipeline (illustrative architecture).

Implementation steps:

1. Create an Azure Storage Account and a Linux VM. Install Python, PySpark, and the necessary dependencies on the VM.
2. Write ingest.py to download the taxi dataset and retrieve weather data from the API. Save the original inputs to raw/.
3. Write transform.py in PySpark to read the raw datasets, convert types, filter invalid records, write staged data, aggregate daily taxi statistics, join the weather data by date, and save a curated Parquet result.
4. Write validate.py to verify required fields, valid counts and precipitation values, and uniqueness of dates in the curated result.
5. Write run_pipeline.py to execute the steps in the correct order, stopping if a required validation fails. Validation should take place before the final result is published.
6. Provide a Spark SQL query that reads the curated Parquet dataset and returns daily trip counts alongside precipitation.

The scripts may use temporary local files during processing, while the required raw, staged, and curated datasets remain in Azure Blob Storage.

Execution:

```
python run_pipeline.py
```

The script coordinates ingestion, transformations, validation, and writing the final results. Re-running the pipeline for the same input period should replace the corresponding output rather than duplicate it.

Live demonstration: Start the pipeline on the prepared VM using a small input dataset or previously downloaded raw inputs. Show processing logs, curated files, and an example SQL query.

What this demonstrates: A complete data engineering workflow without Airflow, Azure Data Factory, Databricks, containers, or Infrastructure as Code.

Example B — Managed Cloud Data Pipeline

Approach: Implement the same solution using managed data engineering services and automated workflow execution.

Technologies:

- Azure Data Lake Storage Gen2 for storage.
- Azure Databricks for PySpark processing.
- Delta Lake for staged and curated tables.
- Databricks Jobs (Lakeflow Jobs) for orchestration.
- Spark SQL or Databricks SQL for querying.

Architecture:

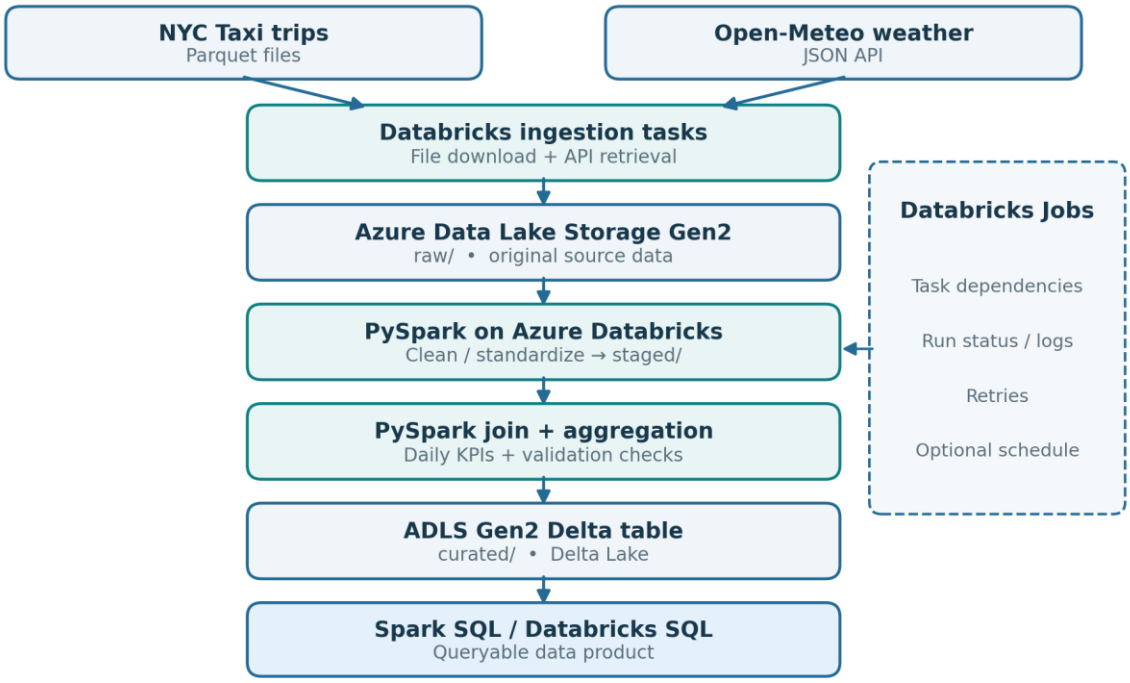


Figure 2. Managed Azure / Databricks pipeline (illustrative architecture).

Implementation steps:

1. Create an ADLS Gen2 storage account and an Azure Databricks workspace.
2. Implement ingestion tasks for taxi and weather data, preserving original inputs in the raw layer.
3. Create PySpark notebooks or Python tasks to clean and standardize the datasets, storing results as Delta tables in the staged layer.
4. Aggregate taxi activity by date and join the result with weather observations.
5. Add data quality checks that identify invalid records and prevent publishing unacceptable results.
6. Store the final dataset as a Delta table using an appropriate overwrite or upsert strategy to avoid duplicates when reprocessing the same dates.
7. Configure a Databricks Job with task dependencies: ingestion, staging and validation, curated processing and validation, publication.

8. Add suitable retries for transient failures and use execution logs to diagnose unsuccessful tasks. Scheduled execution is optional.
9. Make the curated dataset accessible using Spark SQL or Databricks SQL, depending on the available workspace configuration.

Execution: Trigger the pipeline from the Databricks Jobs interface, where task dependencies, execution status, and failures can be observed. It may be triggered manually during the demonstration.

Live demonstration: Trigger the existing job, show its task execution graph, inspect the final Delta table, and run a SQL query. Explain how invalid data and repeated execution are handled.

What this demonstrates: The same data engineering lifecycle implemented with managed processing and orchestration, improved automation, and more advanced data management features.

Comparing the Two Approaches

Aspect	Example A: Script-Based	Example B: Managed
Processing	PySpark on a VM	PySpark on Databricks
Storage	Azure Blob Storage	ADLS Gen2
Pipeline execution	Python script	Databricks Job
Curated format	Parquet	Delta Lake
SQL access	Spark SQL	Spark SQL / Databricks SQL
Monitoring	Script logs	Job execution history
Infrastructure	Few services, manual VM setup	Managed services, more configuration

Important: Example B is not automatically worth a higher grade than Example A.

A reliable, well-structured, clearly documented script-based pipeline can receive an excellent grade. More advanced cloud services are useful when they solve a specific problem, not simply because they are more sophisticated.

You may implement a simpler, more advanced, or entirely different architecture, provided it satisfies the project requirements and is feasible within your available resources and cloud credits.

4. Deliverables

Submit a **Git repository** containing the following.

Implementation

- Data ingestion and PySpark processing code.
- Data quality checks.
- Scripts or configuration files needed to run the pipeline.
- A small sample dataset, if useful and permitted by the data source.

README

- Project goal and data sources.
- A simple architecture/data flow diagram showing both sources and all three data layers.
- Main technologies and services used.
- Instructions for configuring and running the pipeline.
- Required credentials or configuration variables, without including actual secrets.
- Instructions for stopping or deleting cloud resources to avoid unnecessary costs.

Data Product Documentation

- Description of the final curated dataset.
- Its schema and the meaning of important fields.
- Its intended update frequency, if applicable.
- The quality checks applied.
- At least one example SQL query illustrating how the data product can be used.

Keep the documentation concise and practical. It should help another student understand and reproduce the solution, rather than serve as a comprehensive technical report.

5. Minimum Requirements

To receive a passing grade, your project must demonstrate:

1. Ingestion of data from at least two sources.
2. A working raw → staged → curated data flow.
3. At least one meaningful PySpark transformation.
4. Basic data quality validation.
5. At least one SQL-queryable curated dataset combining information from the sources.
6. A working demonstration and the required documentation.
7. Sufficient personal understanding of the submitted solution.

A simple implementation that reliably meets these requirements is preferable to a complex but incomplete one.

Going Beyond the Minimum

For higher grades, we look for engineering quality and a deeper understanding of the design.

Possible improvements include:

- Automated orchestration or scheduling of the pipeline.
- More robust error handling and retry mechanisms.
- Better data quality validation and test coverage.
- More sophisticated transformations or analytical outputs.
- Improved logging, monitoring, or metadata management.
- Careful handling of incremental data updates.

These are **examples, not an additional mandatory checklist**. Select improvements that make sense for your project. Adding technologies without a clear purpose does not automatically improve your grade.

6. Evaluation

The final grade is based on four areas:

Area	Weight	What we evaluate
Data Pipeline & Data Product	40%	Correct ingestion, processing, integration, and a useful, queryable result
Data Quality & Reliability	20%	Meaningful validation, repeatability, and sensible error handling
Documentation & Reproducibility	15%	Clear architecture, justified choices, and practical execution instructions
Demonstration & Understanding	25%	A working demonstration and the ability to explain and defend the implementation

Mandatory Understanding Requirement

Demonstrated understanding is mandatory, regardless of the technical quality of the submitted project.

During the demonstration, you must be able to explain the essential parts of your solution, including:

- How data moves through the pipeline.
- What the main transformations do and why they are needed.
- How data from the two sources is combined.
- What the data quality checks verify.
- How the final dataset is produced and queried.
- Why the main technical choices were made.

The instructors may also ask you to inspect, modify, or troubleshoot a small part of your implementation, such as adjusting a data quality rule, changing a SQL query, or explaining a section of your PySpark code.

Questions may also cover any topic taught during the semester, even if it is unrelated to the technologies used in your project. These questions will focus on fundamental concepts and principles rather than detailed, tool-specific knowledge. You are expected to understand the core topics covered in the course, regardless of which technologies you chose for your implementation.

The following grading limits apply:

- If your demonstration does not establish sufficient personal understanding of the submitted work, your final grade may be limited to **2 (pass)**, regardless of technical quality or documentation.
- If you cannot explain or operate the essential parts of your solution, the project may receive **1 (fail)**.

These decisions are based on **demonstrated understanding**, not simply on whether AI tools or existing examples were used.

7. Practical Information

Collaboration: Discussing ideas and technologies with other students is encouraged, but the submitted implementation must represent your own work unless group work is explicitly authorized.

AI tools: AI assistants and existing code examples are allowed. You are responsible for understanding, validating, and explaining the resulting solution.

Cloud costs: Use small datasets and cost-effective resources. Stop or delete unused cloud services. Avoid relying on paid services or subscription features beyond your available credits. Contact the instructors if access or credit limitations prevent you from using the intended environment.

Deadline: December 18, 2026.

Demonstration: Presentation timeslots will be announced during the semester. The cloud environment may be prepared in advance; only the data pipeline and its results need to be demonstrated live.

Consultation: Contact the instructors if you need advice on selecting a dataset, designing the architecture, or resolving technical difficulties.